Chuleta de examen – Juego de sumas (Unidad 3)

Cada cambio dice **dónde va** (función + línea exacta del código original), **qué escribir** y, si hace falta, **qué más adaptar**. Los números de línea son los del código original (último commit con código `c457f89`). **Se mueven en cuanto añades código**, así que busca el texto entre comillas con **Ctrl+F**. Al final del PDF tienes el código original **con números de línea**.

Cómo trabajar en el examen

1. Abre el juego con **Live Server** y la consola a la vez (**F12 → Consola**).
2. Haz un cambio cada vez: guarda, pulsa **F5**, prueba y solo entonces sigue con el siguiente.
3. La partida queda guardada en `sessionStorage`. Para volver a ver el formulario, pulsa **Nueva partida**.
4. Si al recargar se pierde la partida, la culpa es de `cargarPartida`, que rechaza los datos guardados que ya no encajan con tu cambio.

> ⚠ **TRAMPA – `cargarPartida` (líneas 217–223)**: solo acepta partidas con **puntos enteros**, con **`puntos === historial.length`** y con **`numero1`/`numero2` del 1 al 10**. Si cambias los puntos por acierto, pones puntos decimales, restas puntos o cambias el rango, **adapta esas líneas**. Si no, al pulsar F5 vuelve el formulario. `cargarRecord` (línea 272) también exige `Number.isInteger(guardado)`, así que rechaza récords con decimales.

Mapa del código (juego.js original)

Línea	Función / dato	Qué hace
4–23	`const ... = document.getElementById(...)`	Elementos del HTML
25	`EDAD_MINIMA = 12`	Edad mínima
29–38	`let`	nombreJugador, edadJugador, numero1, numero2, resuelta, puntuacion, record, historial
40	`calcularEdad`	Edad con Date (resta 1 si no ha cumplido)
55	`cambiarColorPanel`	Azul si ≥18, verde si no
65	`mostrarPanel`	Nombre en mayúsculas y edad
76	`validarPerfil`	Valida el formulario, “Acceso denegado”/“permitido”
120	`siguienteMision`	Genera la suma con Math.random
138	`comprobarRespuesta`	Compara (línea 152), historial, +1 punto, récord
175	`abrirAyuda`	window.open + document.write
192 / 208	`guardarPartida` / `cargarPartida`	sessionStorage `partidaSumas`
253	`reiniciarPartida`	removeItem + reload
269 / 281	`cargarRecord` / `actualizarRecord`	localStorage `recordSumas` + Math.max
303 / 311 / 323	`aplicarLetra` / `cargarPreferencia` / `guardarPreferencia`	Cookie `letraSumas` 30 días

index.html: línea 11 es el `<script ... defer>`, la 15 el `<h1>`, la 22 el `</select>`, la 35 “Debes tener al menos 12 años”, la 63 el botón Ayuda y la 67 `<p id="historial">`.

---

1. Mostrar info por consola

La consola se ve en **F12 → Consola**, no en la página. `console.log` sirve para datos normales, `console.warn` para avisos, `console.error` para errores y `console.table` para arrays y objetos.

Edad del jugador: en `mostrarPanel`, **debajo de la línea 71** `textoEdad.textContent = "Edad: " ...`:

```
        console.log("Edad del jugador:", edadJugador);
```

Acceso permitido: en `validarPerfil`, **debajo de la línea 114** `mensaje.textContent = "Acceso permitido. Perfil creado.";`:

```
        console.log("Acceso permitido para", nombre);
```

Acceso denegado: ⚠ **no** lo pongas debajo de la línea 85 (`"Acceso denegado"`), porque esa línea se ejecuta **siempre**: la función empieza denegando. Ponlo **antes de cada `return;`** de `validarPerfil` (**líneas 88, 102 y 107**):

```
        console.error("Acceso denegado");
        return;
```

Números generados: en `siguienteMision`, **debajo de la línea 129** `textoSuma.textContent = ...`:

```
        console.log(`Nueva suma: ${numero1} + ${numero2} = ${numero1 + numero2}`);
```

Acierto y fallo: en `comprobarRespuesta`. Las dos primeras líneas van **debajo de la línea 161** (`"¡Correcto! ..."`):

```
        console.log("Acierto:", numero1 + " + " + numero2 + " = " + respuesta);
        console.table(historial);                // el array como tabla
```

Y esta **debajo de la línea 165** (`"No es correcto. ..."`):

```
        console.warn("Fallo: la respuesta correcta era", numero1 + numero2);
```

Lo guardado en sessionStorage: en `guardarPartida`, **debajo de la línea 200** `sessionStorage.setItem(...)`:

```
        console.log("Guardado:", sessionStorage.getItem("partidaSumas"));
        console.table(partida);                  // o el objeto como tabla
```

Errores al guardar o leer: dentro de los `catch (error) {` (**líneas 202, 248, 258, 275 y 290**): `console.error(error);`

Cookies: en `cargarPreferencia`, **debajo de la línea 320** `aplicarLetra();`: `console.log("Cookies:", document.cookie);`

Otros: `console.info("...")`, `console.group("Misión"); ... console.groupEnd();`, `console.count("aciertos")`, `console.time("t"); ... console.timeEnd("t")` y `console.clear()`.

---

2. Redondear la puntuación (Math)

Método	2.5	2.4	-2.5	Qué hace
`Math.round(x)`	3	2	-2	al más cercano
`Math.floor(x)`	2	2	-3	hacia abajo
`Math.ceil(x)`	3	3	-2	hacia arriba
`Math.trunc(x)`	2	2	-2	quita los decimales
`x.toFixed(2)`	“2.50”	“2.40”	“-2.50”	**devuelve String** (solo para mostrar)

Para redondear a 2 decimales y seguir teniendo un número: `Math.round(x * 100) / 100`.

Puntos con bonus aleatorio redondeado a 1 decimal: en `comprobarRespuesta`, **sustituye las líneas 159 y 160** (`puntuacion = puntuacion + 1;` y la de `textoPuntuacion`):

```
        const bonus = Math.random();                                  // 0 … 0.999
        puntuacion = Math.round((puntuacion + 1 + bonus) * 10) / 10;   // 1 decimal
        textoPuntuacion.textContent = "Puntos: " + puntuacion.toFixed(1);
```

y la línea 161 (el mensaje):

```
        mensajeJuego.textContent = "¡Correcto! +" + (1 + bonus).toFixed(1) + " puntos.";
```

Adapta también: - `cargarPartida`, **línea 219**: cambia `!Number.isInteger(partida.puntos)` por `typeof partida.puntos !== "number"`. - **Línea 223**: borra `partida.puntos !== partida.historial.length` y deja `!Array.isArray(partida.historial)) {` al final de la línea 222. - `cargarRecord`, **línea 272**: cambia `Number.isInteger(guardado)` por `!isNaN(guardado)`.

Puntos según la dificultad (las sumas grandes dan más): sustituye la línea 159 por `puntuacion = puntuacion + Math.ceil((numero1 + numero2) / 5);` y quita la condición de la línea 223.

Otro redondeo para los números del 1 al 10 (líneas 127–128):

```
        numero1 = Math.ceil(Math.random() * 10);       // ceil hacia arriba
        numero1 = Math.round(Math.random() * 9) + 1;   // round al más cercano
```

---

3. Fechas (Date)

Métodos de Date: - `getFullYear()`: el año. - `getMonth()`: el mes, **de 0 a 11** (suma 1 para mostrarlo). - `getDate()`: el día del mes. - `getDay()`: el día de la semana, **0 = domingo**. - `getHours()`, `getMinutes()`, `getSeconds()`: hora, minutos y segundos. - `getTime()`: milisegundos desde 1970. - `toLocaleDateString("es-ES")` y `toLocaleTimeString("es-ES")`: fecha y hora con formato español.

Mostrar la fecha actual. Son dos partes. - En `index.html`, **debajo de la línea 15** `<h1>Juego de misiones</h1>`:

```
        <p id="fecha-actual"></p>
```

- **Al final de `juego.js`**:

```
const textoFecha = document.getElementById("fecha-actual");
function mostrarFechaActual() {
    const ahora = new Date();
    textoFecha.textContent = "Hoy es " + ahora.toLocaleDateString("es-ES", {
        weekday: "long", day: "numeric", month: "long", year: "numeric"
    });
}
mostrarFechaActual();
```

Formato manual `dd/mm/aaaa` (para meterlo dentro de la función, en lugar de la línea de `toLocaleDateString`):

```
const dia = String(ahora.getDate()).padStart(2, "0");
const mes = String(ahora.getMonth() + 1).padStart(2, "0");   // ¡+1!
textoFecha.textContent = "Hoy es " + dia + "/" + mes + "/" + ahora.getFullYear();
```

Reloj que se actualiza cada segundo (al final de `juego.js`, tras lo anterior):

```
setInterval(function () {
    textoFecha.textContent = new Date().toLocaleString("es-ES");   // fecha y hora
}, 1000);
```

Hora de cada acierto en el historial: en `comprobarRespuesta`, **sustituye la línea 156**:

```
const hora = new Date().toLocaleTimeString("es-ES");
historial.push(numero1 + " + " + numero2 + " = " + respuesta + " (" + hora + ")");
```

Día de la semana y año en que nació: en `validarPerfil`, **debajo de la línea 105** `const edad = calcularEdad(nacimiento, hoy);`:

```
const diasSemana = ["domingo", "lunes", "martes", "miércoles", "jueves", "viernes", "sábado"];
console.log("Naciste un " + diasSemana[nacimiento.getDay()] + " del año " + nacimiento.getFullYear());
```

Días que faltan para su cumpleaños: va en el mismo sitio (debajo de la línea 105):

```
let cumple = new Date(hoy.getFullYear(), nacimiento.getMonth(), nacimiento.getDate());
if (cumple < hoy) {
    cumple.setFullYear(hoy.getFullYear() + 1);
}
const dias = Math.round((cumple - hoy) / (1000 * 60 * 60 * 24));
console.log("Faltan " + dias + " días para tu cumpleaños");
```

Edad mínima: cambia la **línea 25** `const EDAD_MINIMA = 14;` y la **línea 35 de index.html** (“al menos 14 años”). **Edad máxima**: en la **línea 106**, cambia `if (edad < EDAD_MINIMA) {` por `if (edad < EDAD_MINIMA || edad > 99) {`.

---

4. Modificar el color de fondo y otros estilos

Se cambian con `elemento.style.propiedad = "valor"` (en camelCase): `backgroundColor`, `color`, `borderColor`, `fontSize`. - `document.body` cambia **toda la página**. - `document.querySelector("main")` cambia **la caja blanca**. - `panelJugador` cambia **solo el perfil**.

Menor de edad: fondo de la página rojo y aviso por consola. Sustituye `cambiarColorPanel` entera (**líneas 55–63**):

```
function cambiarColorPanel() {
    if (edadJugador >= 18) {
        panelJugador.style.backgroundColor = "#dbeafe";   // Azul claro.
        console.log("Es mayor de edad");
    } else {
        panelJugador.style.backgroundColor = "#dcfce7";   // Verde claro.
        document.body.style.backgroundColor = "#fecaca";  // Página rojo claro.
        console.log("Es menor de edad");
    }
}
```

Tres tramos de edad, color del texto y borde según el nombre (otra versión de la misma función):

```
function cambiarColorPanel() {
    if (edadJugador >= 18) {
        panelJugador.style.backgroundColor = "#dbeafe";   // Azul: mayor de edad.
        panelJugador.style.color = "#1e3a8a";             // Texto azul oscuro.
    } else if (edadJugador >= 15) {
        panelJugador.style.backgroundColor = "#fef9c3";   // Amarillo: 15-17.
    } else {
        panelJugador.style.backgroundColor = "#dcfce7";   // Verde: 12-14.
    }
    if (nombreJugador.length > 6) {
        panelJugador.style.borderColor = "red";           // Borde según el nombre.
    }
}
```

Fondo verde al acertar y rojo al fallar: en `comprobarRespuesta`, la primera línea va **debajo de la línea 161** y la segunda **debajo de la línea 165**:

```
        document.body.style.backgroundColor = "#bbf7d0";   // acierto
        document.body.style.backgroundColor = "#fecaca";   // fallo
```

Puntos en verde al llegar a 5: **debajo de la línea 161**:

```
        if (puntuacion >= 5) {
            textoPuntuacion.style.color = "green";
        }
```

Color aleatorio (por ejemplo **debajo de la línea 129**, en cada suma nueva):

```
        const r = Math.floor(Math.random() * 256);
        const g = Math.floor(Math.random() * 256);
        const b = Math.floor(Math.random() * 256);
        document.body.style.backgroundColor = "rgb(" + r + ", " + g + ", " + b + ")";
```

Selector de color del usuario. Son dos partes. - En `index.html`, **debajo de la línea 22** (el `</select>` del tamaño de letra):

```
        <label for="color-fondo">Color de fondo</label>
        <input id="color-fondo" type="color" value="#eef2f6">
```

- **Al final de `juego.js`**:

```
        const selectorColor = document.getElementById("color-fondo");
        selectorColor.addEventListener("input", function () {
            document.body.style.backgroundColor = selectorColor.value;
            localStorage.setItem("colorFondo", selectorColor.value);   // opcional: recordarlo
        });
        const colorGuardado = localStorage.getItem("colorFondo");
        if (colorGuardado) {
            document.body.style.backgroundColor = colorGuardado;
            selectorColor.value = colorGuardado;
        }
```

Con una clase CSS. En `estilos.css` añade `.oscuro { background-color: #111827; color: white; }` y en JS: `document.body.classList.toggle("oscuro");` (también existen `add`, `remove` y `contains`).

---

5. Obtener y modificar elementos del DOM

Obtener	Devuelve
`document.getElementById("mensaje")`	el elemento con ese id
`document.querySelector("#panel-jugador h2")`	el **primero** que cumple el selector CSS
`document.querySelectorAll("button")`	todos los que lo cumplen (se recorren con `forEach`)
`document.getElementsByTagName("p")` / `getElementsByClassName("x")`	colecciones

Modificar	Para qué
`el.textContent = "..."`	texto (seguro, no interpreta HTML)
`el.innerHTML = "<b>..</b>"`	HTML (nunca con datos que escribe el usuario)
`el.style.color = "red"`	estilos
`el.classList.add/remove/toggle("x")`	clases
`el.setAttribute("title", "...")` / `getAttribute`	atributos
`el.hidden = true`	ocultar o mostrar
`input.value`, `boton.disabled = true`	formularios
`document.createElement("p")`, `padre.appendChild(el)`, `el.remove()`	crear y quitar elementos

Cambiar el título con el nombre: en `mostrarPanel`, **debajo de la línea 68**:

```
        document.querySelector("h1").textContent = "Juego de " + nombreJugador;
        document.title = "Juego de " + nombreJugador;       // título de la pestaña
```

Longitud del nombre en el panel: **sustituye la línea 68**:

```
        textoNombre.textContent = "Jugador: " + nombreJugador.toUpperCase() + " (" + nombreJugador.length + " letras)";
```

Historial como lista `<ul>` en lugar de `join`: - En `index.html`, **sustituye la línea 67** por `<ul id="historial"></ul>`. - En `juego.js`, **al final**:

```
function pintarHistorial() {
    textoHistorial.innerHTML = "";              // vaciar la lista
    historial.forEach(function (suma) {
        const li = document.createElement("li");
        li.textContent = suma;
        textoHistorial.appendChild(li);
    });
}
```

- **Sustituye las líneas 157 y 240** (`textoHistorial.textContent = historial.join(" | ");`) por `pintarHistorial();`.

Cambiar todos los botones a la vez: al final de `juego.js`:

```
document.querySelectorAll("#zona-juego button").forEach(function (boton) {
    boton.style.backgroundColor = "#16a34a";
});
```

Botón nuevo "Pista" (el resultado es par o impar): - En `index.html`, **debajo de la línea 63** (el botón Ayuda):

```
<button id="boton-pista" type="button">Pista</button>
```

- **Al final de `juego.js`**:

```
const botonPista = document.getElementById("boton-pista");
function mostrarPista() {
    // % da el resto de dividir: 0 es par, 1 es impar.
    if ((numero1 + numero2) % 2 === 0) {
        mensajeJuego.textContent = "Pista: el resultado es par.";
    } else {
        mensajeJuego.textContent = "Pista: el resultado es impar.";
    }
}
botonPista.addEventListener("click", mostrarPista);
```

Receta para cualquier botón: botón con su `id` en el HTML → `getElementById` → función → `addEventListener("click", función)`.

Botón creado solo con JavaScript (sin tocar el HTML), al final de `juego.js`:

```
const botonBorrar = document.createElement("button");
botonBorrar.type = "button";
botonBorrar.textContent = "Borrar récord";
botonBorrar.addEventListener("click", function () {
    localStorage.removeItem("recordSumas");
    record = 0;
    textoRecord.textContent = "Récord: 0";
});
zonaJuego.appendChild(botonBorrar);
```

Eventos: `click`, `submit` (con `evento.preventDefault()`), `change`, `input`, `keydown`, `load`, `DOMContentLoaded`, `message` y `storage`.

---

6. Cambiar entre localStorage y sessionStorage

	`sessionStorage`	`localStorage`	**Cookie**
Dura	mientras la pestaña siga abierta	hasta que se borre	hasta `max-age`
Ámbito	solo esa pestaña	todas las pestañas del sitio	se envía al servidor
En tu juego	`partidaSumas`	`recordSumas`	`letraSumas`

Todos usan los mismos métodos: `setItem(clave, texto)`, `getItem(clave)`, `removeItem(clave)` y `clear()`. **Solo guardan texto**: los objetos se pasan con `JSON.stringify` / `JSON.parse` y los números con `String` / `Number`. Para verlo: **F12 → Aplicación → Almacenamiento de sesión / Almacenamiento local / Cookies.**

Que la partida siga al cerrar el navegador (de session a local): cambia `sessionStorage` por `localStorage` en **3 líneas**: - **Línea 200** (`guardarPartida`): `localStorage.setItem("partidaSumas", ...)` - **Línea 210** (`cargarPartida`): `localStorage.getItem("partidaSumas")` - **Línea 256** (`reiniciarPartida`): `localStorage.removeItem("partidaSumas")` - Y el mensaje de la **línea 201**: `"Partida guardada en este navegador."`

Récord solo durante la sesión (de local a session): cambia `localStorage` por `sessionStorage` en **2 líneas**: - **Línea 271** (`cargarRecord`): `sessionStorage.getItem("recordSumas")` - **Línea 288** (`actualizarRecord`): `sessionStorage.setItem("recordSumas", ...)`

Truco: declara una variable arriba (debajo de la línea 25), `const almacen = localStorage;`, y usa `almacen.setItem(...)`. Así solo cambias una palabra.

Guardar el último jugador: en `validarPerfil`, **debajo de la línea 111** `nombreJugador = nombre;`:

```
localStorage.setItem("ultimoJugador", nombre);
```

Para rellenar el campo al abrir la página, al final de `juego.js`:

```
        campoNombre.value = localStorage.getItem("ultimoJugador") || "";
```

Borrar todo: `localStorage.clear(); sessionStorage.clear();`

Cookies: - **Cookie de 7 días**: en la **línea 327** cambia `max-age=2592000` por `max-age=604800` (7 × 24 × 60 × 60) y en la línea 336 pon `"... durante 7 días."`. - **Borrar la cookie**: `document.cookie = "letraSumas=; max-age=0; path=/";` - **Tercer tamaño de letra**: - En el HTML, debajo de la línea 21: `<option value="enorme">Enorme</option>`. - En `aplicarLetra`, la **línea 306** `} else {` pasa a ser `} else if (selectorLetra.value === "enorme") { document.body.style.fontSize = "24px"; } else {`. - En `cargarPreferencia`, la **línea 316** pasa a ser `if (cookies[i].trim() === "letraSumas=grande" || cookies[i].trim() === "letraSumas=enorme") {` y la 317 a `selectorLetra.value = cookies[i].trim().split("=")[1];`.

---

7. Mensajes entre ventanas (postMessage)

- **Enviar**: `otraVentana.postMessage(dato, origen)`.
- **Recibir**: `window.addEventListener("message", function (evento) { ... evento.data ... })`.
- **De la principal a la ayuda**: `ventana.postMessage(...)`, usando la referencia que devuelve `window.open`.
- **De la ayuda a la principal**: `window.opener.postMessage(...)`.
- **Seguridad**: comprueba siempre `evento.origin === window.location.origin`.

Más datos en la ventana de ayuda (sin postMessage): - **Línea 177**: `window.open("", "_blank", "width=300,height=250")` cambia el tamaño. - **Sustituye las líneas 186–187**:

```
        ventana.document.getElementById("datos").textContent = "Jugador: " + nombreJugador +
            ". Edad: " + edadJugador + ". Puntos: " + puntuacion + ". Sumas: " + historial.join(", ") + ".";
```

Ayuda que se actualiza en directo y responde con postMessage. Son tres partes.

Parte 1: sustituye `abrirAyuda` entera (líneas 175–188) por:

```
        let ventanaAyuda = null;   // referencia para enviar mensajes después

        function abrirAyuda() {
            ventanaAyuda = window.open("", "_blank", "width=450,height=400");
            if (!ventanaAyuda) {
                mensajeJuego.textContent = "Permite las ventanas emergentes para ver la ayuda.";
                return;
            }
            ventanaAyuda.document.write('<!DOCTYPE html><html lang="es"><head><meta charset="UTF-8"><title>Ayuda</title></head>
<h1>Cómo jugar</h1><p id="datos"></p><button id="saludar">Enviar mensaje al juego</button></body></html>');
            ventanaAyuda.document.close();

            // La ayuda RECIBE mensajes de la principal
            ventanaAyuda.addEventListener("message", function (evento) {
                if (evento.origin !== window.location.origin) return;
                ventanaAyuda.document.getElementById("datos").textContent =
                    "Jugador: " + evento.data.nombre + ". Puntos: " + evento.data.puntos;
            });
            // La ayuda ENVÍA un mensaje a la principal
            ventanaAyuda.document.getElementById("saludar").addEventListener("click", function () {
                ventanaAyuda.opener.postMessage("¡Hola desde la ayuda!", window.location.origin);
            });
            enviarDatosAyuda();
        }

        function enviarDatosAyuda() {
            if (ventanaAyuda && !ventanaAyuda.closed) {
                ventanaAyuda.postMessage({ nombre: nombreJugador, puntos: puntuacion }, window.location.origin);
            }
        }

        // La principal RECIBE mensajes de la ayuda
        window.addEventListener("message", function (evento) {
            if (evento.origin !== window.location.origin) return;
            if (typeof evento.data === "string") {
                mensajeJuego.textContent = "Mensaje recibido: " + evento.data;
            }
        });
```

Parte 2: en `comprobarRespuesta`, **debajo de la línea 162** `actualizarRecord();`:

```
        enviarDatosAyuda();
```

Parte 3: la línea 190 (`botonAyuda.addEventListener("click", abrirAyuda);`) se queda igual.

Versión con un archivo `ayuda.html` propio: - En `abrirAyuda`: `ventanaAyuda = window.open("ayuda.html", "_blank", "width=450,height=400");` (quita el `document.write` y el `close`). - Al final de `juego.js`:

```
window.addEventListener("message", function (e) {
    if (e.origin !== window.location.origin) return;
    if (e.data === "lista") enviarDatosAyuda();     // la ayuda avisa cuando ha cargado
});
```

- `ayuda.html`:

```
<p id="datos"></p>
<script>
  window.addEventListener("message", function (e) {
      if (e.origin !== window.location.origin) return;
      document.getElementById("datos").textContent =
          "Jugador: " + e.data.nombre + ". Puntos: " + e.data.puntos;
  });
  window.opener.postMessage("lista", window.location.origin);
</script>
```

(El aviso “lista” hace falta porque si envías antes de que `ayuda.html` cargue, el mensaje se pierde.)

Entre pestañas: - **Evento** `storage`: salta en las **otras** pestañas cuando cambia localStorage.

```
window.addEventListener("storage", function (e) {
    if (e.key === "recordSumas") textoRecord.textContent = "Récord: " + e.newValue;
});
```

- **BroadcastChannel**:

```
const canal = new BroadcastChannel("sumas");
canal.postMessage({ puntos: puntuacion });
canal.onmessage = function (e) { console.log("Otra pestaña:", e.data); };
```

- **Cerrar la ayuda**: `ventanaAyuda.close()`. Para saber si está cerrada: `ventanaAyuda.closed`.

---

8. Cambiar las reglas del juego

2 puntos por acierto: hay que tocar tres líneas. - **Línea 159**: `puntuacion = puntuacion + 2;` - **Línea 161**: `mensajeJuego.textContent = "¡Correcto! Has ganado 2 puntos. Pulsa Siguiente.";` - **Línea 223** (`cargarPartida`): `partida.puntos !== partida.historial.length * 2) {`

Restar 1 punto al fallar, sin bajar de 0: - En `comprobarRespuesta`, **sustituye la línea 165** por:

```
puntuacion = Math.max(0, puntuacion - 1);       // nunca negativo
textoPuntuacion.textContent = "Puntos: " + puntuacion;
mensajeJuego.textContent = "No es correcto. Pierdes 1 punto.";
guardarPartida();
```

- En `cargarPartida`, **líneas 222–223**:

```
// ANTES
typeof partida.resuelta !== "boolean" || !Array.isArray(partida.historial) ||
    partida.puntos !== partida.historial.length) {
// DESPUÉS
typeof partida.resuelta !== "boolean" || !Array.isArray(partida.historial)) {
```

Números del 1 al 20: cambia `* 10` por `* 20` en las **líneas 127 y 128**, y en `cargarPartida` cambia `> 10` por `> 20` en las **líneas 220 y 221**. Fórmula general: `Math.floor(Math.random() * (max - min + 1)) + min`.

Multiplicar en vez de sumar: hay que tocar cuatro sitios. - **Línea 129** y **línea 237**: `numero1 + " x " + numero2 + " = ?"` - **Línea 152**: `if (respuesta === numero1 * numero2) {` - **Línea 156**: `historial.push(numero1 + " x " + numero2 + " = " + respuesta);`

Límite de 3 intentos: - Debajo de la línea 38: `let intentos = 3;` - En `siguienteMision`, debajo de la línea 131 (`resuelta = false;`): `intentos = 3;` - **Sustituye la línea 165**:

```
intentos = intentos - 1;
if (intentos === 0) {
    mensajeJuego.textContent = "Sin intentos. Era " + (numero1 + numero2) + ". Pulsa Siguiente.";
    botonComprobar.disabled = true;
} else {
    mensajeJuego.textContent = "Fallo. Te quedan " + intentos + " intentos.";
}
```

10 segundos por suma: - Debajo de la línea 38: `let temporizador;` - En `siguienteMision`, debajo de la línea 134 (`campoRespuesta.focus();`):

```
        clearTimeout(temporizador);
        temporizador = setTimeout(function () {
            mensajeJuego.textContent = "¡Tiempo agotado! Pulsa Siguiente.";
            botonComprobar.disabled = true;
        }, 10000);
```

- Debajo de la línea 153 (`resuelta = true;`): `clearTimeout(temporizador);`

Ganar al llegar a 10 puntos: debajo de la línea 161:

```
        if (puntuacion >= 10) {
            mensajeJuego.textContent = "¡Has ganado!";
            botonSiguiente.disabled = true;
        }
```

Confirmar antes de reiniciar: en `reiniciarPartida`, **debajo de la línea 253**: `if (!confirm("¿Seguro que quieres empezar de nuevo?")) return;`

Aviso de nuevo récord: en `actualizarRecord`, **debajo de la línea 283** `cargarRecord();`:

```
        if (puntuacion > record) {
            console.log("¡Nuevo récord!");
        }
```

Nombre de al menos 3 letras: en `validarPerfil`, **línea 87**: `if (nombre.length < 3 || fecha === "" || !campoFecha.validity.valid) {`

Contador de fallos que se conserva al recargar: hay que tocar cuatro sitios. - Debajo de la línea 35: `let fallos = 0;` - **Sustituye la línea 165**:

```
        fallos = fallos + 1;
        mensajeJuego.textContent = "No es correcto. Fallos: " + fallos;
        guardarPartida();
```

- **Línea 196**: `numero1: numero1, numero2: numero2, resuelta: resuelta, historial: historial, fallos: fallos`
- **Debajo de la línea 233** `historial = partida.historial;`: `fallos = Number(partida.fallos) || 0;`

---

9. Métodos de String, Date, Math, Number y Array

Tipo	Método	Qué hace
String	`length`	número de letras
String	`trim()`	quita los espacios de los extremos
String	`toUpperCase()` / `toLowerCase()`	mayúsculas / minúsculas
String	`charAt(0)` / `slice(1)`	primera letra / desde la posición 1
String	`includes("a")` / `indexOf("a")`	si contiene / en qué posición
String	`split(";")` / `replace("a","b")` / `padStart(2,"0")`	separar / reemplazar / rellenar
Date	`getFullYear()` `getMonth()` `getDate()` `getDay()`	año / mes 0-11 / día / día de la semana 0-6
Math	`random()`	decimal de 0 a menos de 1
Math	`floor()` `ceil()` `round()` `trunc()`	redondeos
Math	`max()` `min()` `abs()` `pow(a,b)` `sqrt()`	mayor / menor / sin signo / potencia / raíz
Number	`Number("5")` / `parseInt("12px")` / `parseFloat`	texto → número
Number	`Number.isInteger(5.5)` → false / `isNaN(x)`	comprueba si es entero / si no es un número
Number	`x.toFixed(2)`	2 decimales (String)
Array	`push(x)` / `pop()`	añade al final / quita el último
Array	`length` / `[0]` / `[length - 1]`	tamaño / primero / último
Array	`join(", ")` / `forEach(f)` / `includes(x)`	une en texto / recorre / si contiene

Primera letra mayúscula y el resto minúscula (en `mostrarPanel`, sustituyendo `nombreJugador.toUpperCase()` en la línea 68): `nombreJugador.charAt(0).toUpperCase() + nombreJugador.slice(1).toLowerCase()`

Última suma resuelta: debajo de la línea 156: `console.log("Última suma:", historial[historial.length - 1]);` ⚠ Si quitas elementos del historial (por ejemplo con `pop`), recuerda que la línea 223 compara los puntos con `historial.length`.

---

10. Quitar `defer` (cargar el script sin defer)

Son dos cambios:

```
<!-- index.html, línea 11: antes era <script src="juego.js" defer></script> -->
<script src="juego.js"></script>
```

```
// juego.js, debajo de la línea 1 "use strict";
window.addEventListener("load", function () {
    // ... TODO el código que ya tenías ...
});   // ← esta línea va al FINAL del archivo
```

Por qué: sin `defer` el script se ejecuta antes de que existan los elementos. Entonces `getElementById` devuelve `null` y sale el error *Cannot read properties of null (reading 'addEventListener')*. También vale `document.addEventListener("DOMContentLoaded", function () {`, que salta cuando el HTML está listo sin esperar a las imágenes. Otra opción: mover el `<script src="juego.js"></script>` al final del `<body>`.

---

11. Preguntas de justificación

Pregunta	Respuesta
¿Por qué `defer`?	El script está en el `head`; `defer` espera a que el HTML esté listo para que `getElementById` encuentre los elementos.
¿Para qué `preventDefault()`?	Evita que el formulario recargue la página al enviarse.
¿Para qué `novalidate`?	Para validar con JS y mostrar nuestro "Acceso denegado" en vez del aviso del navegador.
¿Para qué `trim()`?	Quita los espacios de los extremos: un nombre de solo espacios queda vacío y se rechaza.
¿Por qué `return` en `validarPerfil`?	Termina la función para que no siga con datos incorrectos.
¿Cómo se calcula la edad?	Se restan los años con `getFullYear`; si el cumpleaños de este año no ha llegado (comparando `getMonth` y `getDate`), se resta uno.
¿Por qué `getMonth` cuenta desde 0?	En Date, enero es 0 y diciembre es 11.
¿Por qué `textContent` y no `innerHTML`?	`textContent` escribe texto sin interpretar HTML, así que lo que escriba el jugador no se ejecuta.
¿Cómo salen números del 1 al 10?	`Math.random()` da un decimal de 0 a menos de 1. Al multiplicar por 10 y aplicar `Math.floor` salen enteros de 0 a 9; con `+ 1`, de 1 a 10.
¿Para qué `resuelta`?	Impide dar el punto dos veces por la misma suma (por ejemplo, pulsando Enter dos veces o al recargar).
¿Para qué `Number.isInteger`?	Después de `Number(...)`, rechaza respuestas decimales o vacías.
¿Para qué `Math.max` en el récord?	Conserva el mayor entre el récord anterior y los puntos actuales.
¿Por qué se vuelve a leer el récord antes de guardarlo?	Por si otra pestaña lo ha mejorado mientras tanto.
¿Diferencia entre sessionStorage, localStorage y cookie?	sessionStorage dura mientras la pestaña siga abierta; localStorage sigue al cerrar el navegador; la cookie tiene caducidad (`max-age`) y viaja al servidor en las peticiones.
¿Para qué `JSON.stringify` / `JSON.parse`?	El almacenamiento solo guarda texto: `stringify` convierte el objeto en texto y `parse` lo recupera.
¿Para qué `try/catch` al guardar?	El navegador puede bloquear el almacenamiento; así se avisa y se puede seguir jugando.
¿Qué devuelve `window.open`?	La ventana nueva, o `null` si el navegador la bloquea.
¿Para qué `postMessage`?	Para enviar datos entre ventanas de forma segura, comprobando `evento.origin`.
¿Diferencia entre `let` y `const`?	`const` no se puede reasignar; `let` sí.
¿Por qué `===`?	Compara el valor **y** el tipo (`"5" === 5` es false).
¿Para qué `"use strict"`?	Obliga a declarar las variables y detecta más errores.
¿Por qué Live Server?	Las cookies necesitan HTTP; con `file://` no funcionan bien.

Anexo – Código original con números de línea

index.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <!-- Adapta el ancho de la página a la pantalla del móvil o del ordenador. -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Juego de misiones - Unidad 3</title>
    <link rel="stylesheet" href="estilos.css">
    <!-- defer espera a que el HTML esté preparado antes de ejecutar JavaScript.
         Así podemos acceder al formulario y al párrafo aunque el script esté en head. -->
    <script src="juego.js" defer></script>
</head>
<body>
    <main>
        <h1>Juego de misiones</h1>
        <p>Resuelve sumas sencillas y consigue puntos.</p>

        <label for="tamano-letra">Tamaño de letra</label>
        <select id="tamano-letra">
            <option value="normal">Normal</option>
            <option value="grande">Grande</option>
        </select>
        <p id="mensaje-preferencia" aria-live="polite"></p>

        <!-- Cada label se une a su campo mediante for e id.
             required indica que el campo no puede quedar vacío. -->
        <!-- novalidate deja la validación a JavaScript para mostrar nuestro mensaje. -->
        <form id="formulario-jugador" novalidate>
            <label for="nombre">Nombre del jugador</label>
            <input id="nombre" type="text" required>

            <label for="fecha-nacimiento">Fecha de nacimiento</label>
            <input id="fecha-nacimiento" type="date" required>

            <p>Debes tener al menos 12 años para jugar.</p>
            <button id="boton-inicio" type="submit">Comenzar</button>
        </form>

        <!-- JavaScript cambiará este mensaje al pulsar el botón.
             aria-live permite que los lectores de pantalla anuncien el cambio. -->
        <p id="mensaje" aria-live="polite">Introduce tus datos y pulsa Comenzar.</p>

        <!-- hidden oculta el panel hasta que el jugador introduce datos válidos. -->
        <section id="panel-jugador" hidden>
            <h2>Tu perfil</h2>
            <p id="nombre-jugador"></p>
            <p id="edad-jugador"></p>
        </section>
        <!-- El juego solo se muestra después de validar el perfil. -->
        <section id="zona-juego" hidden>
            <h2>Resuelve la suma</h2>
            <p id="puntuacion" aria-live="polite">Puntos: 0</p>
            <p id="record" aria-live="polite">Récord: 0</p>
            <p id="mensaje-record" aria-live="polite"></p>
            <p id="suma"></p>
            <form id="formulario-respuesta" novalidate>
                <label for="respuesta">Tu respuesta</label>
                <input id="respuesta" type="number" step="1" required>
                <button id="boton-comprobar" type="submit">Comprobar</button>
                <button id="boton-siguiente" type="button">Siguiente</button>
            </form>
            <p id="mensaje-juego" aria-live="polite"></p>
            <button id="boton-ayuda" type="button">Ayuda</button>
            <button id="boton-reiniciar" type="button">Nueva partida</button>
            <p id="mensaje-guardado" aria-live="polite"></p>
            <h3>Sumas resueltas</h3>
            <p id="historial">Todavía no has resuelto ninguna suma.</p>
        </section>
    </main>
</body>
</html>
```

juego.js

```javascript
"use strict";

// defer hace que este código se ejecute después de crear los elementos del HTML.
```

```
const formularioJugador = document.getElementById("formulario-jugador");
const campoNombre = document.getElementById("nombre");
const campoFecha = document.getElementById("fecha-nacimiento");
const mensaje = document.getElementById("mensaje");
const panelJugador = document.getElementById("panel-jugador");
const textoNombre = document.getElementById("nombre-jugador");
const textoEdad = document.getElementById("edad-jugador");
const zonaJuego = document.getElementById("zona-juego");
const textoSuma = document.getElementById("suma");
const formularioRespuesta = document.getElementById("formulario-respuesta");
const campoRespuesta = document.getElementById("respuesta");
const botonComprobar = document.getElementById("boton-comprobar");
const botonSiguiente = document.getElementById("boton-siguiente");
const mensajeJuego = document.getElementById("mensaje-juego");
const textoHistorial = document.getElementById("historial");
const textoPuntuacion = document.getElementById("puntuacion");
const botonAyuda = document.getElementById("boton-ayuda");
const botonReiniciar = document.getElementById("boton-reiniciar");
const mensajeGuardado = document.getElementById("mensaje-guardado");
const textoRecord = document.getElementById("record");
const mensajeRecord = document.getElementById("mensaje-record");
const EDAD_MINIMA = 12;

// Guardaremos estos datos solo cuando el perfil sea válido.
// let permite cambiar su valor; const se utiliza para valores que no reasignamos.
let nombreJugador = "";
let edadJugador = 0;
let numero1 = 0;
let numero2 = 0;
let resuelta = false;
let puntuacion = 0;
let record = 0;

// Un array guarda varios datos. Aquí añadiremos las sumas acertadas.
let historial = [];

function calcularEdad(nacimiento, hoy) {
    // Primero restamos los años. Luego comprobamos si ya ha cumplido años.
    let edad = hoy.getFullYear() - nacimiento.getFullYear();
    const mesActual = hoy.getMonth();
    const mesNacimiento = nacimiento.getMonth();

    // Si su cumpleaños todavía no ha llegado, restamos un año.
    if (mesActual < mesNacimiento ||
        (mesActual === mesNacimiento && hoy.getDate() < nacimiento.getDate())) {
        edad = edad - 1;
    }

    return edad;
}

function cambiarColorPanel() {
    // style.backgroundColor cambia el fondo del panel desde JavaScript.
    // Usamos un if/else sencillo: azul para mayores de edad y verde para menores.
    if (edadJugador >= 18) {
        panelJugador.style.backgroundColor = "#dbeafe"; // Azul claro.
    } else {
        panelJugador.style.backgroundColor = "#dcfce7"; // Verde claro.
    }
}

function mostrarPanel() {
    // toUpperCase es un método de String: convierte el nombre a mayúsculas.
    // textContent muestra texto, sin interpretar lo escrito como código HTML.
    textoNombre.textContent = "Jugador: " + nombreJugador.toUpperCase();

    // La edad ya se calculó con los métodos de Date en calcularEdad.
    textoEdad.textContent = "Edad: " + edadJugador + " años";
    cambiarColorPanel();
    panelJugador.hidden = false; // Hace visible el panel.
}

function validarPerfil(evento) {
    // Evita que el formulario recargue la página al pulsar Comenzar o Enter.
    evento.preventDefault();

    // trim elimina los espacios de los extremos: "   " se convierte en "".
    const nombre = campoNombre.value.trim();
    const fecha = campoFecha.value;

    // Empezamos denegando el acceso. Solo lo permitimos si pasan todas las pruebas.
    mensaje.textContent = "Acceso denegado";

    if (nombre === "" || fecha === "" || !campoFecha.validity.valid) {
        return; // Termina la función: no seguimos con un dato vacío o incorrecto.
```

```
    }

    // El campo date devuelve año-mes-día. Añadimos la hora para usar la fecha local.
    const nacimiento = new Date(fecha + "T00:00:00");
    const hoy = new Date();
    hoy.setHours(0, 0, 0, 0); // Quitamos la hora para comparar solo los días.

    // isNaN detecta una fecha que JavaScript no ha podido interpretar.
    // La comparación del día también rechaza fechas como el 30 de febrero,
    // que JavaScript puede convertir automáticamente en un día de marzo.
    const diaEscrito = Number(fecha.slice(8, 10));
    if (isNaN(nacimiento.getTime()) || nacimiento.getDate() !== diaEscrito ||
        nacimiento >= hoy) {
        return; // Rechazamos fechas imposibles, la fecha de hoy y fechas futuras.
    }

    const edad = calcularEdad(nacimiento, hoy);
    if (edad < EDAD_MINIMA) {
        return; // Debe haber cumplido al menos 12 años.
    }

    // Solo llegamos aquí si todos los datos son correctos.
    nombreJugador = nombre;
    edadJugador = edad;
    formularioJugador.hidden = true;
    mensaje.textContent = "Acceso permitido. Perfil creado.";
    mostrarPanel();
    zonaJuego.hidden = false;
    siguienteMision();
}

function siguienteMision() {
    if (nombreJugador === "") {
        return; // No generamos una misión sin un perfil válido.
    }

    // random da un decimal desde 0 hasta menos de 1.
    // Multiplicamos por 10, floor quita los decimales y +1 da un número del 1 al 10.
    numero1 = Math.floor(Math.random() * 10) + 1;
    numero2 = Math.floor(Math.random() * 10) + 1;
    textoSuma.textContent = numero1 + " + " + numero2 + " = ?";
    campoRespuesta.value = "";
    resuelta = false; // Esta nueva suma todavía no se ha acertado.
    botonComprobar.disabled = false;
    mensajeJuego.textContent = "";
    campoRespuesta.focus();
    guardarPartida();
}

function comprobarRespuesta(evento) {
    evento.preventDefault();

    if (nombreJugador === "" || resuelta) {
        return; // Una suma ya acertada no se puede registrar dos veces.
    }

    // value es texto. Number lo convierte a un número para comparar la respuesta.
    const respuesta = Number(campoRespuesta.value);
    if (campoRespuesta.value.trim() === "" || !Number.isInteger(respuesta)) {
        mensajeJuego.textContent = "Escribe un número entero.";
        return;
    }

    if (respuesta === numero1 + numero2) {
        resuelta = true;
        botonComprobar.disabled = true;
        // push añade un dato al final del array; join une los datos para mostrarlos.
        historial.push(numero1 + " + " + numero2 + " = " + respuesta);
        textoHistorial.textContent = historial.join(" | ");
        // Solo sumamos aquí: el control de resuelta impide repetir el punto.
        puntuacion = puntuacion + 1;
        textoPuntuacion.textContent = "Puntos: " + puntuacion;
        mensajeJuego.textContent = "¡Correcto! Has ganado 1 punto. Pulsa Siguiente.";
        actualizarRecord();
        guardarPartida();
    } else {
        mensajeJuego.textContent = "No es correcto. Inténtalo otra vez.";
    }
}

// La función se ejecuta al enviar el formulario, no al cargar la página.
formularioJugador.addEventListener("submit", validarPerfil);

formularioRespuesta.addEventListener("submit", comprobarRespuesta);
botonSiguiente.addEventListener("click", siguienteMision);
```

```
174
175 function abrirAyuda() {
176     // Abrimos la ventana directamente desde el clic para evitar bloqueos.
177     const ventana = window.open("", "_blank", "width=450,height=400");
178     if (!ventana) {
179         mensajeJuego.textContent = "Permite las ventanas emergentes para ver la ayuda.";
180         return;
181     }
182
183     // El HTML es fijo. El nombre se escribe después con textContent, de forma segura.
184     ventana.document.write('<!DOCTYPE html><html lang="es"><head><meta charset="UTF-8">
title>Ayuda</title></head><body><h1>Cómo jugar</h1><p id="datos"></p><p>Suma los dos números y pulsa Comprobar. Cada
cierto da 1 punto.</p><p>Si fallas, puedes volver a intentarlo. Siguiente muestra otra suma.</p><p>Cada suma solo puede
ar un punto. Puedes cerrar esta ventana para seguir jugando.</p></body></html>');
185     ventana.document.close();
186     ventana.document.getElementById("datos").textContent =
187         "Jugador: " + nombreJugador + ". Puntos actuales: " + puntuacion + ".";
188 }
189
190 botonAyuda.addEventListener("click", abrirAyuda);
191
192 function guardarPartida() {
193     // Un objeto agrupa los datos de la partida con nombres fáciles de reconocer.
194     const partida = {
195         nombre: nombreJugador, edad: edadJugador, puntos: puntuacion,
196         numero1: numero1, numero2: numero2, resuelta: resuelta, historial: historial
197     };
198     try {
199         // sessionStorage guarda texto en esta pestaña. JSON.stringify convierte el objeto a texto.
200         sessionStorage.setItem("partidaSumas", JSON.stringify(partida));
201         mensajeGuardado.textContent = "Partida guardada en esta pestaña.";
202     } catch (error) {
203         // Si el navegador bloquea el almacenamiento, se puede seguir jugando.
204         mensajeGuardado.textContent = "No se ha podido guardar la partida.";
205     }
206 }
207
208 function cargarPartida() {
209     try {
210         const guardada = sessionStorage.getItem("partidaSumas");
211         if (!guardada) {
212             return; // Si no hay partida, dejamos visible el formulario inicial.
213         }
214         // JSON.parse convierte el texto guardado otra vez en un objeto.
215         const partida = JSON.parse(guardada);
216         // Comprobaciones básicas para no cargar datos incompletos o dañados.
217         if (!partida || typeof partida.nombre !== "string" || partida.nombre.trim() === "" ||
218             !Number.isInteger(partida.edad) || partida.edad < EDAD_MINIMA ||
219             !Number.isInteger(partida.puntos) || partida.puntos < 0 ||
220             !Number.isInteger(partida.numero1) || partida.numero1 < 1 || partida.numero1 > 10 ||
221             !Number.isInteger(partida.numero2) || partida.numero2 < 1 || partida.numero2 > 10 ||
222             typeof partida.resuelta !== "boolean" || !Array.isArray(partida.historial) ||
223             partida.puntos !== partida.historial.length) {
224             return;
225         }
226
227         nombreJugador = partida.nombre;
228         edadJugador = partida.edad;
229         puntuacion = partida.puntos;
230         numero1 = partida.numero1;
231         numero2 = partida.numero2;
232         resuelta = partida.resuelta;
233         historial = partida.historial;
234         formularioJugador.hidden = true;
235         mostrarPanel();
236         zonaJuego.hidden = false;
237         textoSuma.textContent = numero1 + " + " + numero2 + " = ?";
238         textoPuntuacion.textContent = "Puntos: " + puntuacion;
239         if (historial.length > 0) {
240             textoHistorial.textContent = historial.join(" | ");
241         }
242         // Restaurar resuelta impide ganar otra vez el punto después de recargar.
243         botonComprobar.disabled = resuelta;
244         mensaje.textContent = "Partida recuperada.";
245         if (resuelta) {
246             mensajeJuego.textContent = "Esta suma ya está resuelta. Pulsa Siguiente.";
247         }
248     } catch (error) {
249         mensaje.textContent = "No se pudo recuperar la partida. Introduce tus datos.";
250     }
251 }
252
253 function reiniciarPartida() {
254     try {
255         // Solo borramos nuestra partida, no otros datos de la pestaña.
```

```
        sessionStorage.removeItem("partidaSumas");
        window.location.reload();
    } catch (error) {
        mensajeGuardado.textContent = "No se ha podido borrar la partida guardada.";
    }
}

botonReiniciar.addEventListener("click", reiniciarPartida);
// Se ejecuta al cargar la página: recupera la misma suma sin generar otra.
cargarPartida();

// localStorage conserva el récord aunque cierres el navegador.
// Number convierte el texto guardado a número; rechazamos valores dañados.
function cargarRecord() {
    try {
        const guardado = Number(localStorage.getItem("recordSumas"));
        if (Number.isInteger(guardado) && guardado >= 0) {
            record = Math.max(record, guardado);
        }
    } catch (error) {
        mensajeRecord.textContent = "No se pudo leer el récord guardado.";
    }
    textoRecord.textContent = "Récord: " + record;
}

function actualizarRecord() {
    // Otra pestaña puede haber mejorado el récord: lo leemos antes de guardar.
    cargarRecord();
    // Math.max elige el mayor: una partida con menos puntos no reduce el récord.
    record = Math.max(record, puntuacion);
    textoRecord.textContent = "Récord: " + record;
    try {
        localStorage.setItem("recordSumas", String(record));
        mensajeRecord.textContent = "";
    } catch (error) {
        mensajeRecord.textContent = "El récord solo se conservará mientras esta página siga abierta.";
    }
}

// Se carga después de recuperar la partida para incluir sus puntos en el récord.
cargarRecord();
actualizarRecord();

// La cookie guarda solo una preferencia, nunca el nombre ni la fecha de nacimiento.
const selectorLetra = document.getElementById("tamano-letra");
const mensajePreferencia = document.getElementById("mensaje-preferencia");

function aplicarLetra() {
    if (selectorLetra.value === "grande") {
        document.body.style.fontSize = "20px";
    } else {
        document.body.style.fontSize = "16px";
    }
}

function cargarPreferencia() {
    // document.cookie devuelve las cookies juntas, separadas por punto y coma.
    // split crea un array; trim quita los espacios que separan cada cookie.
    const cookies = document.cookie.split(";");
    for (let i = 0; i < cookies.length; i = i + 1) {
        if (cookies[i].trim() === "letraSumas=grande") {
            selectorLetra.value = "grande";
        }
    }
    aplicarLetra();
}

function guardarPreferencia() {
    aplicarLetra();
    // max-age son segundos: 30 días. path=/ permite leerla en todo este sitio.
    // SameSite=Lax limita el envío de esta cookie desde otros sitios.
    document.cookie = "letraSumas=" + selectorLetra.value + "; max-age=2592000; path=/; SameSite=Lax";
    const cookies = document.cookie.split(";");
    let guardada = false;
    for (let i = 0; i < cookies.length; i = i + 1) {
        if (cookies[i].trim() === "letraSumas=" + selectorLetra.value) {
            guardada = true;
        }
    }
    if (guardada) {
        mensajePreferencia.textContent = "Preferencia guardada durante 30 días.";
    } else {
        mensajePreferencia.textContent = "El navegador no ha guardado la preferencia.";
    }
}
```

```
341
342 selectorLetra.addEventListener("change", guardarPreferencia);
343 cargarPreferencia();
```

estilos.css

```
1  /* Incluye el relleno y el borde dentro del ancho de cada elemento.
2     Esto evita que los elementos ocupen más espacio del esperado. */
3  * {
4      box-sizing: border-box;
5  }
6
7  /* Estilos generales de la página: letra, fondo y separación exterior. */
8  body {
9      margin: 0;
10     padding: 24px;
11     font-family: Arial, sans-serif;
12     background-color: #eef2f6;
13     color: #1f2937;
14 }
15
16 /* Centra el panel y limita su ancho para facilitar la lectura. */
17 main {
18     max-width: 600px;
19     margin: 40px auto;
20     padding: 24px;
21     background-color: white;
22     border-radius: 12px;
23 }
24
25 h1 {
26     margin-top: 0;
27 }
28
29 p {
30     line-height: 1.5;
31 }
32
33 /* El cursor indica que el botón se puede pulsar. */
34 button {
35     padding: 12px 20px;
36     border: none;
37     border-radius: 6px;
38     background-color: #1d4ed8;
39     color: white;
40     font: inherit;
41     cursor: pointer;
42 }
43
44 /* Cambia el color al pasar el ratón por encima del botón. */
45 button:hover {
46     background-color: #1e40af;
47 }
48
49 /* Destaca el botón cuando se selecciona usando el teclado. */
50 button:focus-visible {
51     outline: 3px solid #111827;
52     outline-offset: 3px;
53 }
54
55 /* Coloca cada etiqueta encima de su campo y separa los datos del formulario. */
56 label {
57     display: block;
58     margin-bottom: 6px;
59 }
60
61 input, select {
62     display: block;
63     width: 100%;
64     padding: 10px;
65     margin-bottom: 16px;
66     border: 1px solid #6b7280;
67     border-radius: 6px;
68     font: inherit;
69 }
70
71 /* Separa el panel del mensaje y deja espacio alrededor de los datos. */
72 #panel-jugador {
73     margin-top: 20px;
74     padding: 16px;
75     border: 1px solid #6b7280;
76     border-radius: 6px;
77     overflow-wrap: anywhere; /* Evita que un nombre largo se salga del panel. */
```

```css
}

/* Deja separación entre botones y permite leer un historial largo. */
#zona-juego {
    margin-top: 24px;
    overflow-wrap: anywhere;
}

#zona-juego button {
    margin-bottom: 8px;
}

/* Un botón gris indica que la suma ya se ha acertado. */
button:disabled {
    background-color: #6b7280;
    cursor: default;
}
```